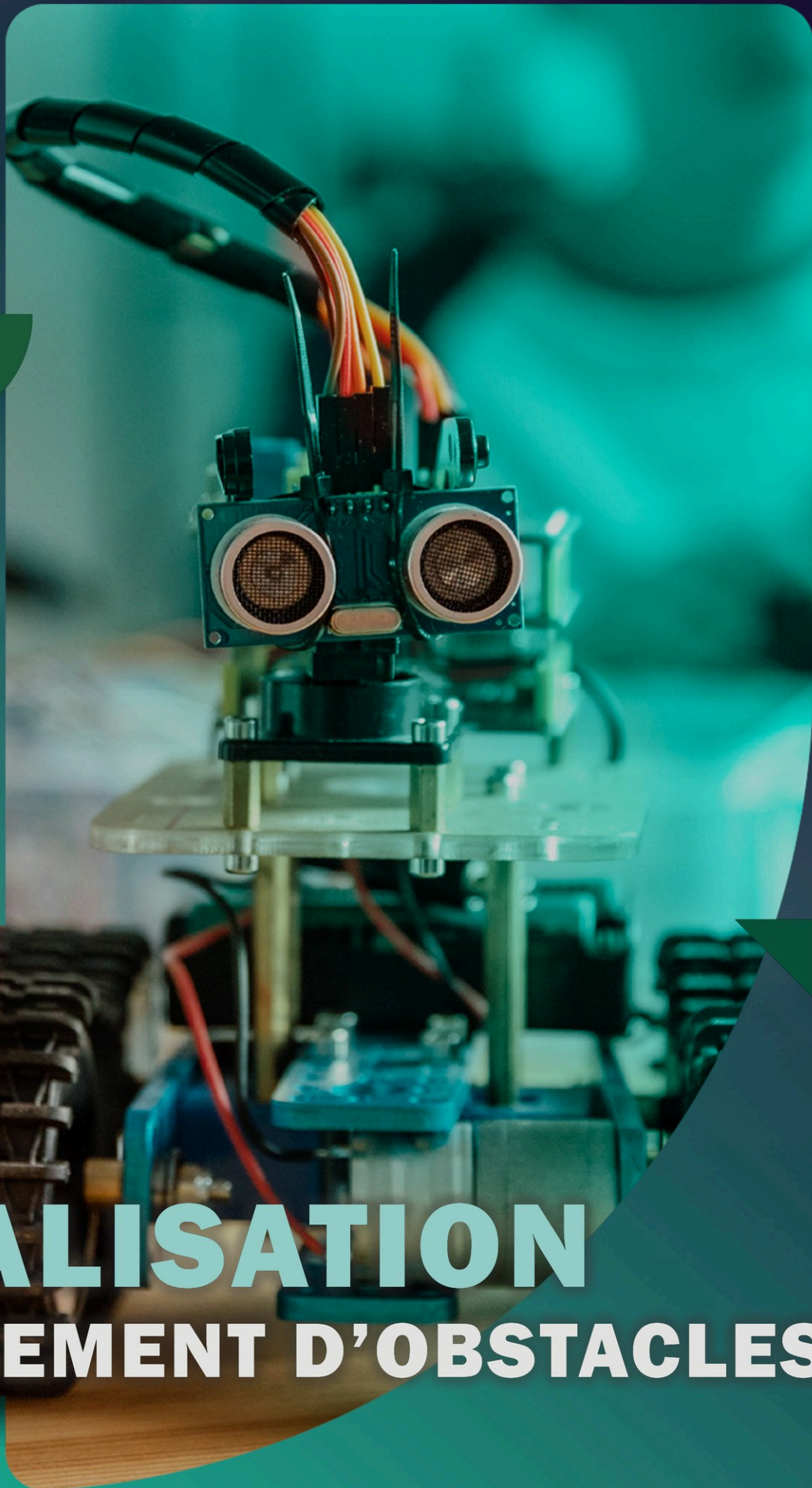




LOCALISATION ET ÉVITEMENT D'OBSTACLES

19
Fev

► Préparé par :
- Aya Briksine

► encadré par :
- M.Dominique Nelson-Gruel

Résumé

Ce projet de Master en robotique et automatisme a pour objectif la conception et la réalisation d'un robot mobile autonome destiné à évoluer dans un environnement intérieur. Le robot est équipé de capteurs lidar et infrarouges permettant la détection et l'évitement d'obstacles, ainsi que la navigation en temps réel dans un espace fermé.

La méthodologie adoptée repose sur une analyse des besoins fonctionnels et techniques, suivie de la définition de l'architecture matérielle et logicielle du système. Le développement comprend l'assemblage du robot, l'intégration des capteurs, la programmation des algorithmes de commande et le traitement des données, ainsi que la réalisation de tests expérimentaux afin de valider la performance et la fiabilité du système.

La navigation en temps réel est assurée par une boucle continue de perception-décision-action : acquisition des données des capteurs, traitement par l'Arduino Uno, puis commande des moteurs pour éviter les obstacles. Toutefois, les ressources limitées de l'Arduino Uno restreignent la complexité des algorithmes embarqués, ce qui limite la précision de la navigation dans des environnements complexes.

Les résultats obtenus montrent que le robot est capable de se déplacer de manière autonome et sécurisée dans un environnement intérieur simple, ouvrant ainsi la voie à des améliorations futures, notamment l'optimisation logicielle et l'ajout de capteurs supplémentaires.

Mots-clés : *Robot mobile autonome, Environnement intérieur, Capteurs infrarouges, Évitement d'obstacles, Système embarqué, Arduino Uno, Temps réel, Robotique et automatisme*

Table des matières

Résumé	2
Introduction Générale	1
1 Étude Fonctionnelle	2
1.1 Introduction	2
1.2 Description du système	2
1.3 Analyse fonctionnelle	2
1.3.1 Diagramme pieuvre	2
1.3.2 Diagramme FAST	3
1.3.3 Grafcet fonctionnel	4
1.4 Conclusion	4
2 Conception et Réalisation	5
2.1 Introduction	5
2.2 Choix matériels	5
2.3 Architecture physique du robot	7
2.3.1 Implantation des composants	7
2.3.2 Système de commande et électronique embarquée	8
2.4 Architecture logicielle	10
3 Tests et Validation	11
3.1 Introduction	11
3.2 Objectifs des tests	11
3.3 Méthodologie des tests	11
3.4 Environnement de test	12
3.5 Résultats obtenus	12
3.5.1 Performances mesurées	13

4	Analyse Critique et Améliorations	14
4.1	Introduction	14
4.2	Analyse critique du système	14
4.3	Améliorations proposées	15
4.3.1	Intégration d'une intelligence décisionnelle	15
4.3.2	Supervision et contrôle à distance	15
5	Utilisation de la STM32 Nucleo F303RE pour les tests de capteurs	16
5.1	Présentation de la carte STM32 Nucleo F303RE	16
5.2	Objectif des tests	17
5.3	Test du LiDAR Mini (TF-Mini)	17
5.3.1	Présentation du capteur	17
5.3.2	Branchement du TF-Mini sur la STM32	18
5.3.3	Format de trame UART	18
5.3.4	Code STM32 — Lecture du TF-Mini (UART)	19
5.3.5	Configuration dans STM32CubeIDE	20
5.3.6	Résultats obtenus	21
5.4	Test du capteur infrarouge	21
5.4.1	Présentation du capteur	21
5.4.2	Branchement du capteur IR sur la STM32	21
5.4.3	Code STM32 — Détection d'obstacle (GPIO)	22
5.4.4	Configuration GPIO dans STM32CubeIDE	23
5.4.5	Résultats obtenus	23
5.5	Bilan des tests	23
	Conclusion Générale	24
	Bibliographie	25

Table des figures

1	Diagramme pieuvre du robot éviteur d'obstacles	2
2	Diagramme FAST du système	3
3	Grafcet fonctionnel du robot	4
4	Emplacement des composants matériels sur la plateforme du robot	7
5	Position de l'Arduino Uno sur le châssis	8
6	Branchement du capteur LiDAR	9
7	Branchement des capteurs infrarouges latéraux	9
8	Branchement des moteurs DC	10
9	Objectifs des tests	11
10	Schéma du terrain de test	12

Liste des tableaux

1	Fonctions du robot	3
2	Composants matériels du robot mobile autonome	6
3	Performances du robot éviteur d'obstacles	13
4	Limitations et solutions adoptées	14
5	Branchement du LiDAR TF-Mini sur la STM32 Nucleo F303RE	18
6	Format de trame UART du TF-Mini	18
7	Branchement du capteur infrarouge sur la STM32 Nucleo F303RE	21
8	Récapitulatif des tests de capteurs sur la STM32 Nucleo F303RE	23

Introduction Générale

Les systèmes embarqués constituent aujourd'hui un domaine en pleine expansion et occupent une place essentielle dans l'évolution technologique moderne. Présents dans de nombreux dispositifs du quotidien, ils permettent d'assurer des fonctions spécifiques tout en respectant des contraintes strictes de temps réel, de consommation énergétique et de ressources matérielles limitées.

Dans le domaine de la robotique mobile, les systèmes embarqués jouent un rôle central en assurant la perception, le traitement de l'information et la commande des actionneurs. Ils permettent notamment la navigation autonome, l'évitement d'obstacles et la prise de décisions en temps réel.

Dans le cadre de ce projet, nous nous intéressons à la conception et à la réalisation d'un robot mobile autonome capable d'évoluer dans un environnement intérieur et d'éviter les obstacles en temps réel. La problématique principale est la suivante :

Comment concevoir un système embarqué efficace et réactif permettant à un robot mobile autonome d'éviter les obstacles en temps réel tout en respectant les contraintes matérielles et de calcul ?

Ce rapport est structuré en quatre chapitres :

- Le **premier chapitre** présente l'étude fonctionnelle du système à travers l'analyse des besoins et la définition des fonctions.
- Le **deuxième chapitre** détaille la conception et la réalisation matérielle du robot.
- Le **troisième chapitre** est consacré aux tests expérimentaux et à la validation des performances.
- Le **quatrième chapitre** propose une analyse critique du système réalisé et des pistes d'amélioration.

1 Étude Fonctionnelle

1.1 Introduction

Une analyse fonctionnelle est nécessaire afin d'exprimer clairement le besoin lié à la conception du robot éviteur d'obstacles. Elle permet de définir les fonctions principales du système ainsi que les contraintes associées.

1.2 Description du système

Le robot évolue dans un environnement intérieur en détectant les obstacles à l'aide de capteurs. Les informations acquises sont traitées par un microcontrôleur qui prend les décisions nécessaires pour assurer une navigation autonome et sécurisée.

1.3 Analyse fonctionnelle

1.3.1 Diagramme pieuvre

Le diagramme pieuvre permet de représenter graphiquement les interactions entre le système et son environnement.

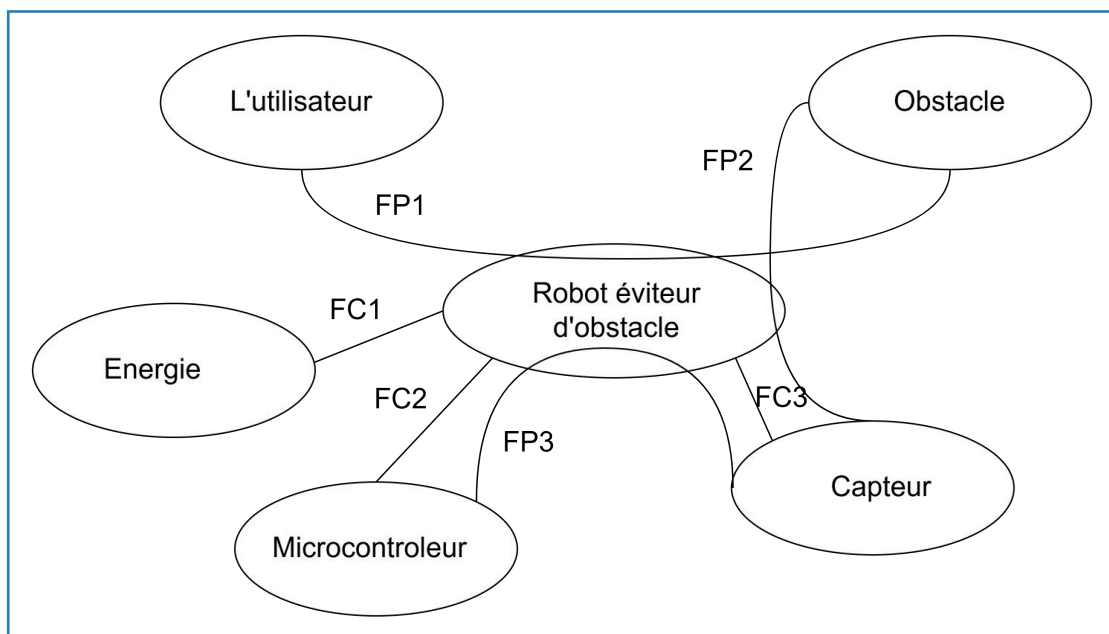


FIGURE 1 – Diagramme pieuvre du robot éviteur d'obstacles

FP : Fonctions principales

FC : Fonctions contraintes

TABLE 1 – Fonctions du robot

Fonction	Description
FP1	Garantir un déplacement autonome et sécurisé
FP2	Détecter la présence d'obstacles
FP3	Éviter les obstacles détectés
FC1	Utiliser une source d'énergie embarquée
FC2	Respecter les contraintes du temps réel

1.3.2 Diagramme FAST

Le diagramme FAST (Function Analysis System Technique) détaille les solutions techniques associées à chaque fonction.

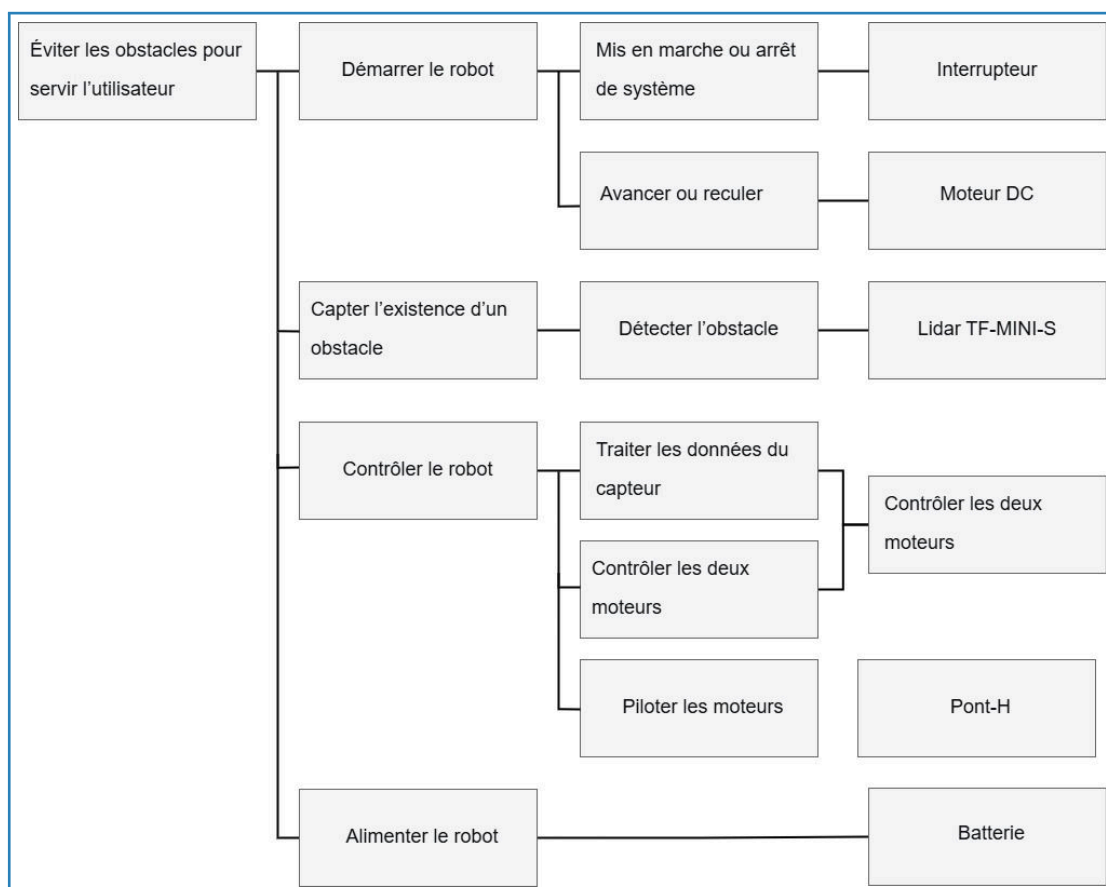


FIGURE 2 – Diagramme FAST du système

1.3.3 Grafcet fonctionnel

À partir des informations fournies ci-dessus, nous avons réalisé le Grafcet de niveau 1 suivant. On définit les variables suivantes :

- **m** : mise en marche du robot
- **o** : présence d'un obstacle
- **d** : probabilité de trouver un obstacle à droite plus élevée qu'à gauche
- **g** : probabilité de trouver un obstacle à gauche plus élevée qu'à droite

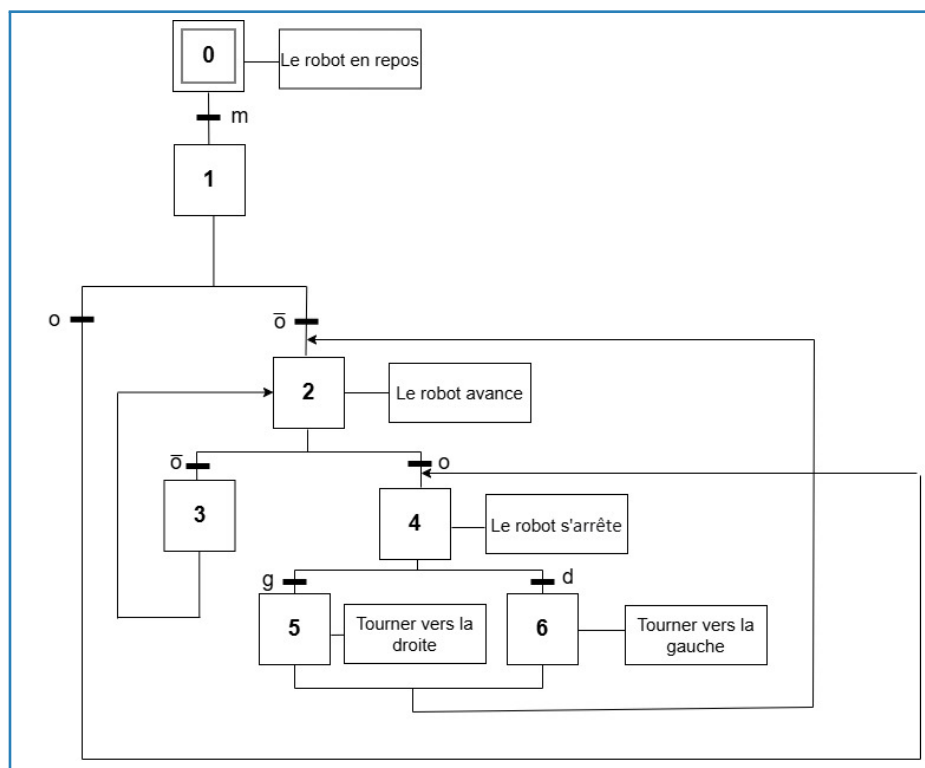


FIGURE 3 – Grafcet fonctionnel du robot

1.4 Conclusion

L'étude fonctionnelle a permis de formaliser les besoins et de définir clairement les spécifications du robot. Les différents diagrammes présentés constituent une base solide pour la phase de conception qui suit.

2 Conception et Réalisation

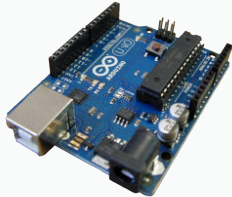
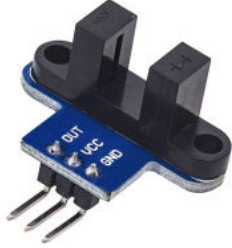
2.1 Introduction

Ce chapitre présente la conception et la réalisation du robot mobile autonome. Il décrit les choix matériels et logiciels, l'intégration des composants et l'architecture électronique embarquée.

2.2 Choix matériels

Cette section présente les différents composants matériels utilisés pour la conception du robot mobile autonome. Chaque élément a été choisi afin d'assurer la mobilité, la perception de l'environnement et le contrôle du robot lors de la navigation en intérieur.

TABLE 2 – Composants matériels du robot mobile autonome

Composant	Justification / Rôle	Photo
AlphaBot	Plateforme mobile compacte et stable, permettant l'intégration facile des capteurs et des moteurs. Choisie pour sa robustesse et sa facilité de manipulation lors des tests de navigation autonome.	
Arduino Uno	Microcontrôleur assurant le traitement des données des capteurs et le contrôle des moteurs en temps réel. Choisi pour sa fiabilité, sa simplicité et sa compatibilité avec les composants du robot.	
TFMini LiDAR	Capteur léger et rapide fournissant des mesures précises pour les obstacles à courte et moyenne distance. Choisi pour sa compacité et sa réactivité en environnement intérieur.	
Capteurs IR latéraux	Capteurs placés à l'arrière pour détecter les obstacles lors des manœuvres de recul, assurant la sécurité du robot.	
Capteur infrarouge de vitesse	Utilisé comme encodeur optique pour mesurer la vitesse et estimer le déplacement du robot à partir du comptage des impulsions.	
Moteurs DC	Assurent le déplacement du robot avec un contrôle indépendant des roues gauche et droite.	

2.3 Architecture physique du robot

2.3.1 Implantation des composants

Les capteurs du robot sont positionnés de manière à assurer une perception efficace de l'environnement et un suivi précis du mouvement.

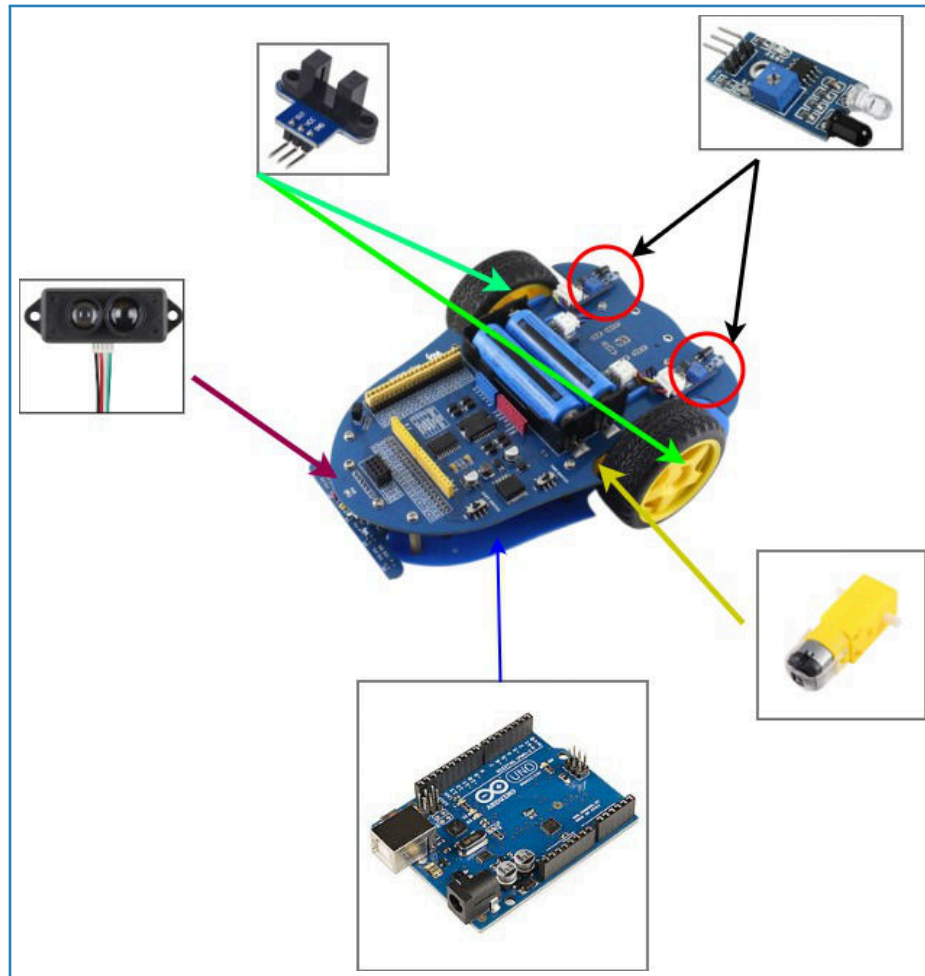


FIGURE 4 – Emplacement des composants matériels sur la plateforme du robot

Le capteur LiDAR TFMini est placé à l'avant du robot afin de détecter les obstacles situés sur la trajectoire de déplacement. Les capteurs infrarouges sont installés à l'arrière pour sécuriser les manœuvres de recul. Les capteurs infrarouges de vitesse, utilisés comme encodeurs optiques, sont positionnés à proximité des roues afin d'estimer la vitesse et le déplacement du robot.

2.3.2 Système de commande et électronique embarquée

Emplacement de l'Arduino sur le châssis L'Arduino Uno est fixé sur la partie centrale du châssis, dans l'emplacement prévu à cet effet. Cette position assure la stabilité du microcontrôleur, protège les composants des chocs et facilite l'accès aux broches pour le branchement des capteurs et des moteurs.

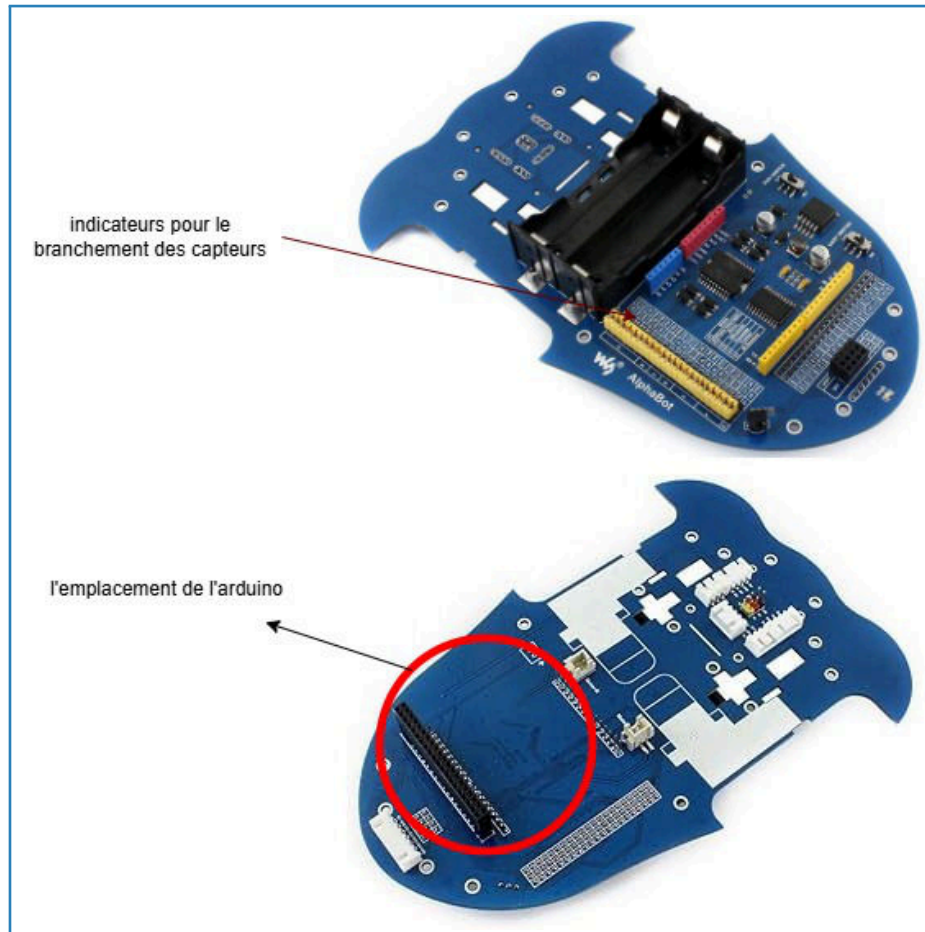


FIGURE 5 – Position de l'Arduino Uno sur le châssis

Branchement du LiDAR Le LiDAR TFMini est connecté à l'Arduino Uno via les broches TX et RX, permettant la transmission en temps réel des mesures de distance.

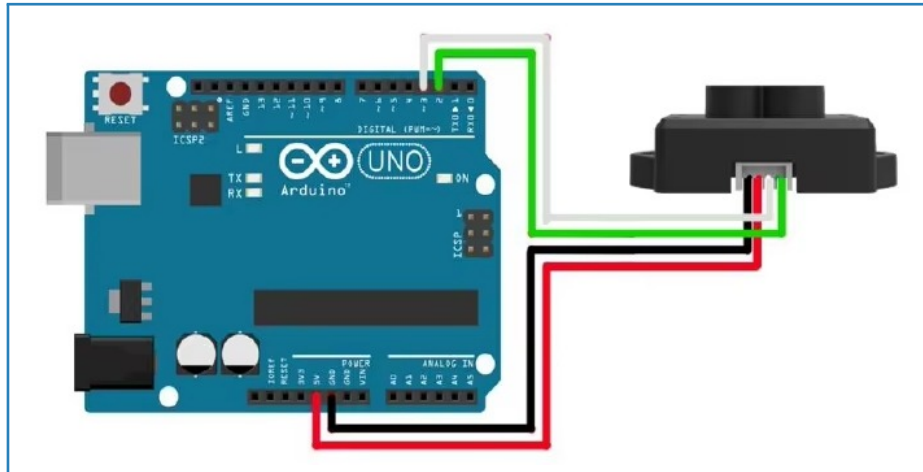


FIGURE 6 – Branchement du capteur LiDAR

Branchement des capteurs infrarouges latéraux Les capteurs infrarouges latéraux sont connectés aux entrées numériques de l'Arduino Uno, permettant la détection des obstacles sur les côtés du robot.

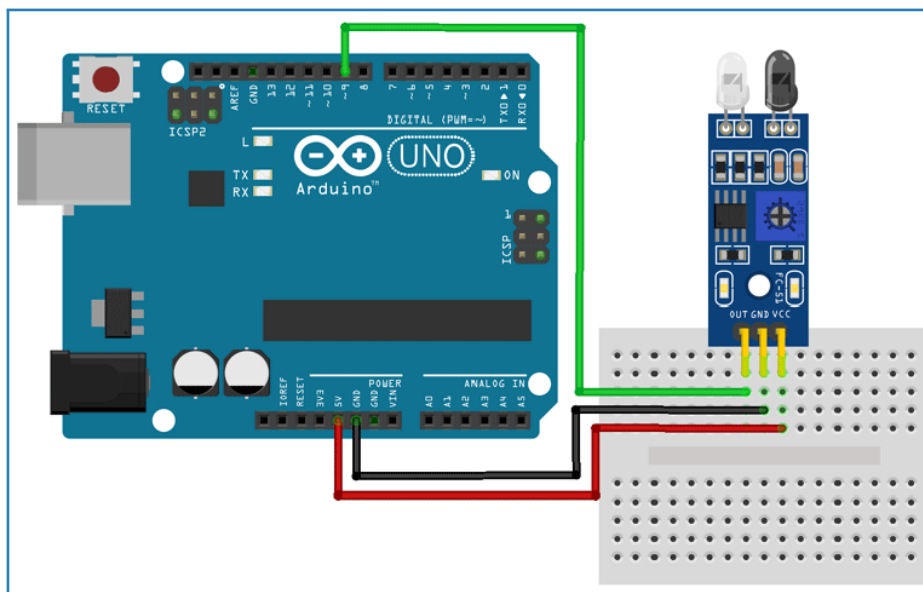


FIGURE 7 – Branchement des capteurs infrarouges latéraux

Branchement des moteurs Les moteurs DC sont connectés à l'Arduino Uno via un driver de moteur, permettant un contrôle indépendant des roues gauche et droite.

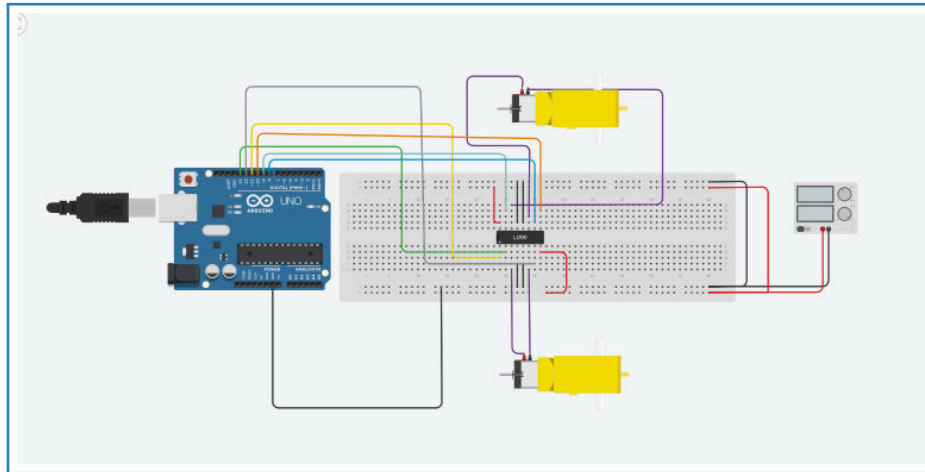


FIGURE 8 – Branchement des moteurs DC

Branchement des capteurs infrarouges de vitesse Les capteurs infrarouges de vitesse, utilisés comme encodeurs optiques, sont placés près des roues du robot et connectés aux broches numériques de l'Arduino Uno.

2.4 Architecture logicielle

Le programme principal est développé sous l'environnement Arduino IDE. Il implémente une boucle de contrôle temps réel qui :

1. Acquiert les données des capteurs (LiDAR et IR)
2. Traite ces informations pour détecter les obstacles
3. Prend une décision sur la trajectoire à suivre
4. Commande les moteurs en conséquence

3 Tests et Validation

3.1 Introduction

Le chapitre de test et validation a pour objectif de vérifier le bon fonctionnement du robot éviteur d'obstacles dans des conditions réelles d'utilisation.

3.2 Objectifs des tests

Les tests réalisés visent à vérifier :

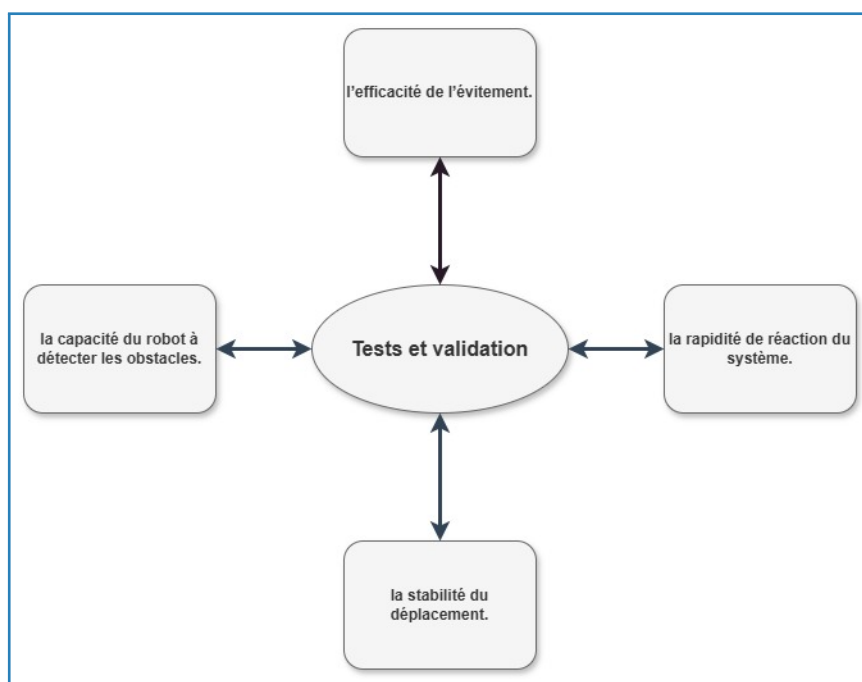


FIGURE 9 – Objectifs des tests

3.3 Méthodologie des tests

Les tests ont été réalisés dans un environnement intérieur simulant des conditions réelles d'utilisation. Dans un premier temps, des simulations ont été effectuées sur la plateforme Tinkercad afin de vérifier le bon fonctionnement du système et d'ajuster les paramètres de commande. Par la suite, nous sommes passés à la mise en œuvre réelle.

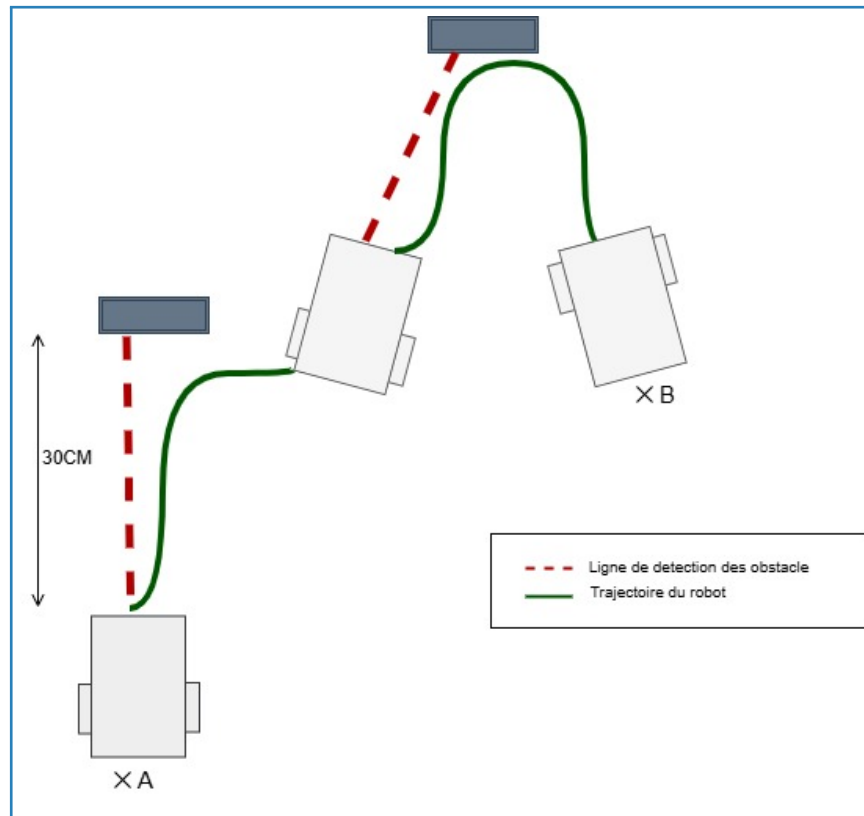


FIGURE 10 – Schéma du terrain de test

3.4 Environnement de test

- Sol plat assurant une bonne stabilité du robot
- Éclairage artificiel et constant
- Obstacles fixes disposés à 30 cm et 40 cm
- Vitesse constante, alimentation stable

3.5 Résultats obtenus

Les résultats confirment le bon fonctionnement du robot. Le capteur Li-DAR assure une détection stable jusqu'à 30 cm. Les capteurs IR latéraux et de vitesse fonctionnent correctement.

3.5.1 Performances mesurées

TABLE 3 – Performances du robot éviteur d'obstacles

Fonction évaluée	Résultat obtenu
Distance de détection	30 cm
Temps de réaction	3 s
Vitesse du robot	150 cm/s
Taux de réussite	90%

4 Analyse Critique et Améliorations

4.1 Introduction

Après la phase de réalisation et de test, il est essentiel d'effectuer une analyse critique du système développé.

4.2 Analyse critique du système

2. Analyse critique du système

Les résultats obtenus montrent que le robot est globalement fonctionnel et capable d'assurer l'évitement autonome des obstacles. L'intégration des capteurs a été réalisée avec succès, permettant une perception efficace de l'environnement.

Le capteur LiDAR assure une détection stable jusqu'à environ 30 cm et permet une vision latérale efficace. Les capteurs infrarouges placés sur les côtés améliorent la détection des obstacles latéraux, tandis que les capteurs infrarouges dédiés à la vitesse permettent de mesurer le déplacement du robot en temps réel.

Cependant, certaines difficultés ont été rencontrées. Le capteur ultrasonique utilisé initialement ne fournissait pas des mesures suffisamment fiables, ce qui a conduit à son remplacement. De plus, l'intégration des capteurs infrarouges latéraux a nécessité plusieurs ajustements, car ils détectaient initialement le sol. Enfin, une différence de portée entre le LiDAR (30 cm) et les capteurs infrarouges (20 cm) a nécessité une calibration spécifique afin d'assurer une détection cohérente.

TABLE 4 – Limitations et solutions adoptées

Limitation	Solution adoptée / proposée
Mesures instables de l'ultrason	Remplacement par un LiDAR plus précis
Détection du sol par IR	Ajustement de l'angle et de la hauteur
Différence de portée (30 cm / 20 cm)	Calibration des seuils de détection
Absence de cartographie	Intégration future d'un algorithme SLAM

4.3 Améliorations proposées

4.3.1 Intégration d'une intelligence décisionnelle

Une amélioration importante consisterait à doter le robot d'un système intelligent capable d'analyser son environnement et d'adapter son comportement en temps réel. Au lieu de simplement éviter les obstacles, le robot pourrait choisir le trajet optimal, anticiper les zones encombrées et améliorer progressivement ses déplacements. Cette évolution transformerait le robot en système autonome et adaptatif.

4.3.2 Supervision et contrôle à distance

Le développement d'une interface de supervision (application mobile ou interface web) permettrait de suivre le fonctionnement du robot en temps réel. L'utilisateur pourrait visualiser les obstacles détectés, surveiller la vitesse du robot et observer son parcours. Cette amélioration rendrait le système connecté, moderne et plus facile à exploiter.

5 Utilisation de la STM32 Nucleo F303RE pour les tests de capteurs

5.1 Présentation de la carte STM32 Nucleo F303RE

La carte **STM32 Nucleo F303RE** est une carte de développement conçue par **STMicroelectronics**. Elle repose sur un microcontrôleur **STM32F303RE**, basé sur un cœur **ARM Cortex-M4 32 bits** cadencé à **72 MHz**, intégrant une unité de calcul en virgule flottante (FPU) qui lui confère d'excellentes performances pour les applications embarquées temps réel.

Caractéristiques techniques principales

- **Processeur** : ARM Cortex-M4 @ 72 MHz avec FPU
- **Mémoire Flash** : 512 KB — **SRAM** : 80 KB
- **Interfaces de communication** : UART, SPI, I2C, CAN, USB
- **Entrées/Sorties** : ADC 12 bits, PWM, GPIO
- **Compatibilité** : broches Arduino Uno et connecteur Morpho ST
- **Alimentation** : via USB (5V) ou source externe (7–12V)
- **Environnement de développement** : STM32CubeIDE, Keil MDK, Arduino IDE (via STM32duino)

Pourquoi utiliser la STM32 ?

La STM32 Nucleo F303RE a été choisie pour sa **polyvalence**, sa **rapidité de prototypage** et sa compatibilité native avec de nombreux capteurs. Elle nous permet de valider le bon fonctionnement des capteurs avant leur intégration définitive dans le robot, en minimisant les risques d'erreurs matérielles ou logicielles en aval.

5.2 Objectif des tests

Nous avons utilisé la STM32 Nucleo F303RE pour tester deux capteurs essentiels au système de perception du robot :

1. **LiDAR TF-Mini** : mesure de distance par temps de vol (ToF)
2. **Capteur infrarouge (IR)** : détection de proximité et d'obstacles

Ces tests ont pour but de :

- Valider la communication entre le capteur et le microcontrôleur
- Vérifier la stabilité, la répétabilité et la précision des mesures
- Préparer l'intégration des capteurs dans le robot final

5.3 Test du LiDAR Mini (TF-Mini)

5.3.1 Présentation du capteur

Le **TF-Mini** est un capteur LiDAR compact développé par **Benewake**, fonctionnant sur le principe du **temps de vol** (Time of Flight). Il émet des impulsions laser infrarouges (850 nm) et mesure le temps de retour du signal réfléchi pour en déduire la distance à un obstacle. Il communique via **UART** et fournit la distance en centimètres.

- **Plage de mesure** : 0,3 m à 12 m
- **Précision** : ± 6 cm (< 6 m) / $\pm 1\%$ (> 6 m)
- **Interface** : UART (115200 baud)
- **Tension d'alimentation** : 5V
- **Consommation** : 130 mA typique
- **Fréquence de trame** : 100 Hz (par défaut)
- **Dimensions** : $42 \times 15 \times 16$ mm

5.3.2 Branchement du TF-Mini sur la STM32

Le TF-Mini dispose de **4 fils** de connexion. Le tableau 5 résume les connexions.

TABLE 5 – Branchement du LiDAR TF-Mini sur la STM32 Nucleo F303RE

Fil TF-Mini	Couleur	Broche STM32	Fonction
VCC	Rouge	5V (CN6)	Alimentation
GND	Noir	GND (CN6)	Masse
TX	Blanc	PA3 (USART2_RX)	Réception données
RX	Vert	PA2 (USART2_TX)	Transmission commandes

Remarque importante : le TF-Mini fonctionne en **5V** mais sa sortie UART est en **3,3V logique**, ce qui est directement compatible avec la STM32. Aucun adaptateur de niveau logique n'est nécessaire.

5.3.3 Format de trame UART

Le TF-Mini envoie une trame de **9 octets** en continu à 100 Hz. La structure de la trame est détaillée dans le tableau 6.

TABLE 6 – Format de trame UART du TF-Mini

Octet	Valeur / Contenu	Description
0	0x59	En-tête 1 (fixe)
1	0x59	En-tête 2 (fixe)
2	Dist_L	Distance — octet faible (cm)
3	Dist_H	Distance — octet fort (cm)
4	Strength_L	Force signal — octet faible
5	Strength_H	Force signal — octet fort
6	Reserved	Réservé
7	Reserved	Réservé
8	Checksum	Somme des octets 0 à 7 (modulo 256)

La distance réelle en centimètres est calculée comme suit :

$$d(cm) = Dist_L + Dist_H \times 256$$

5.3.4 Code STM32 — Lecture du TF-Mini (UART)

Le programme suivant (listing 1) initialise l'UART2, lit les trames, vérifie le checksum et affiche la distance et la force du signal sur la console série (UART1).

```
1  /* Fichier : tfmini.c
2      Description : Lecture de la distance TF-Mini via UART (STM32 HAL
3      )
4      UART2 : PA2(TX) -> RX TF-Mini / PA3(RX) -> TX TF-Mini
5      Baudrate : 115200                                     */
6
7  #include "main.h"
8  #include <stdio.h>
9  #include <string.h>
10
11 extern UART_HandleTypeDef huart2; // UART connect au TF-Mini
12 extern UART_HandleTypeDef huart1; // UART vers PC (moniteur s rie
13                                     )
14
15 #define FRAME_SIZE    9
16 #define HEADER_BYTE   0x59
17
18 /* V rifie l'int grit de la trame par checksum */
19 static uint8_t tfmini_checksum(uint8_t *buf) {
20     uint16_t sum = 0;
21     for (int i = 0; i < FRAME_SIZE - 1; i++) {
22         sum += buf[i];
23     }
24     return ((uint8_t)(sum & 0xFF) == buf[FRAME_SIZE - 1]);
25 }
26
27 /* Lecture en continu du TF-Mini */
28 void tfmini_read(void) {
29     uint8_t byte;
30     uint8_t frame[FRAME_SIZE];
31     uint8_t idx = 0;
32     char    msg[64];
33
34     while (1) {
```

```

33     if (HAL_UART_Receive(&huart2, &byte, 1, 100) != HAL_OK)
34         continue;
35
36     /* Synchronisation sur les deux octets d'entete 0x59 0x59 */
37     if (idx == 0 && byte != HEADER_BYTE) continue;
38     if (idx == 1 && byte != HEADER_BYTE) { idx = 0; continue; }
39
40     frame[idx++] = byte;
41
42     if (idx == FRAME_SIZE) {
43         idx = 0;
44         if (tfmini_checksum(frame)) {
45             uint16_t distance = frame[2] | (frame[3] << 8); //
46                 cm
47             uint16_t strength = frame[4] | (frame[5] << 8); //
48                 u.a.
49             sprintf(msg, "Distance : %d cm | Force signal : %d\r\n",
50                 distance, strength);
51         } else {
52             sprintf(msg, "[ERREUR] Checksum invalide\r\n");
53         }
54         HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg),
55             100);
56     }
57 }

```

Listing 1 – Lecture du TF-Mini sur STM32 (HAL)

5.3.5 Configuration dans STM32CubeIDE

Dans le fichier .ioc, configurer **USART2** comme suit :

- Mode : Asynchronous
- Baud Rate : 115200 bps
- Word Length : 8 bits

- Parity : None
- Stop Bits : 1

5.3.6 Résultats obtenus

Les mesures effectuées sur plusieurs objets à distances connues ont démontré une **bonne stabilité** des lectures et une précision conforme aux spécifications du fabricant. Le capteur répond à 100 Hz sans perte de trame observable. La valeur de force du signal (*strength*) permet en outre de filtrer les mesures peu fiables (signal trop faible sur surfaces très sombres ou très réfléchissantes).

5.4 Test du capteur infrarouge

5.4.1 Présentation du capteur

Le capteur infrarouge utilisé est un module de **détection de proximité** numérique (type FC-51 ou équivalent). Il émet un rayonnement infrarouge et détecte le signal réfléchi par un obstacle. La sortie est **numérique** (signal haut ou bas) selon la présence ou l'absence d'un obstacle dans sa portée réglable.

- **Tension d'alimentation** : 3,3V – 5V
- **Sortie** : numérique (0 = obstacle détecté, 1 = libre)
- **Portée** : 2 cm à 30 cm (réglable via potentiomètre)
- **Interface** : GPIO (digital input)

5.4.2 Branchement du capteur IR sur la STM32

Le tableau 7 détaille les connexions.

TABLE 7 – Branchement du capteur infrarouge sur la STM32 Nucleo F303RE

Broche capteur IR	Broche STM32	Fonction
VCC	3,3V ou 5V (CN6)	Alimentation
GND	GND (CN6)	Masse
OUT	PB4 (D5)	Sortie numérique

5.4.3 Code STM32 — Détection d'obstacle (GPIO)

Le listing 2 présente le code de lecture du capteur IR.

```
1  /* Fichier : ir_sensor.c
2     Description : Lecture du capteur IR via GPIO numérique
3     Broche : PB4 configurée en entrée avec pull-up interne */
4
5  #include "main.h"
6  #include <stdio.h>
7  #include <string.h>
8
9  extern UART_HandleTypeDef huart1; // UART vers PC (moniteur série)
10
11 #define IR_PIN          GPIO_PIN_4
12 #define IR_GPIO_PORT    GPIOB
13
14 void ir_sensor_read(void) {
15     char msg[64];
16
17     while (1) {
18         GPIO_PinState state = HAL_GPIO_ReadPin(IR_GPIO_PORT, IR_PIN);
19
20         if (state == GPIO_PIN_RESET) {
21             // Sortie LOW = obstacle détecté
22             sprintf(msg, ">> Obstacle détecté !\r\n");
23         } else {
24             // Sortie HIGH = voie libre
25             sprintf(msg, "    Voie libre.\r\n");
26         }
27
28         HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), 100);
29         ;
30         HAL_Delay(200); // Lecture toutes les 200 ms
31     }
}
```

Listing 2 – Lecture du capteur IR sur STM32 (HAL)

5.4.4 Configuration GPIO dans STM32CubeIDE

Dans le fichier `.ioc`, configurer la broche **PB4** comme suit :

- Mode : `GPIO_Input`
- Pull-up/Pull-down : `Pull-up`
- Label utilisateur : `IR_SENSOR`

5.4.5 Résultats obtenus

Le capteur a correctement détecté la présence d'obstacles dans sa zone de portée. La réponse est **instantanée** et le signal **stable**, sans rebond significatif. La portée a été ajustée via le potentiomètre intégré au module.

5.5 Bilan des tests

Le tableau 8 récapitule les résultats des tests.

TABLE 8 – Récapitulatif des tests de capteurs sur la STM32 Nucleo F303RE

Capteur	Interface	Stabilité	Validation
LiDAR TF-Mini	UART	Très bonne	
Capteur IR	GPIO numérique	Bonne	

Ces tests confirment que les deux capteurs fonctionnent correctement avec la STM32 et sont prêts à être intégrés dans l'architecture électronique du robot.

Conclusion Générale

Ce projet a permis de concevoir et réaliser un robot mobile autonome éviteur d'obstacles. L'approche méthodique, de l'analyse fonctionnelle aux tests, a abouti à un système fonctionnel avec un taux de réussite de 90%. Les perspectives d'amélioration (intelligence, supervision, SLAM) ouvrent la voie à des applications plus complexes.

Bibliographie

- [1] "Arduino Programming Notebook", Brian W. Evans, 2007
- [2] "Introduction to Autonomous Robots", Nikolaus Correll, 2016
- [3] Documentation technique du capteur TFMMini - Benewake
- [4] "Robotique mobile : des principes à la réalisation", Luc Jaulin, 2015

Annexe

```
1 // Inclure les bibliothèques
2 #include <SoftwareSerial.h>
3 #include "TimerOne.h"
4 #include "TFLidar.h"
5
6 // Encode
7 const byte MOTOR_A = 2;
8 const byte MOTOR_B = 3;
9
10 // Wheel
11 const float stepcount = 20.00;
12 const float wheeldiameter = 66.10;
13 const float wheel_base = 100.0;
14
15 volatile int counter_A = 0;
16 volatile int counter_B = 0;
17
18 // Lidar
19 int lidarDistance = 0;
20
21 // Motor A
22 int enA = 5;
23 int in1 = A0;
24 int in2 = A1;
25
26 // Motor B
27 int enB = 6;
28 int in3 = A2;
29 int in4 = A3;
30
31 enum RobotState {
32     TRAJECTOIRE,
33     EVITEMENT,
34     FIN
35 };
36
37 RobotState currentState = TRAJECTOIRE;
```

```
38 int segment = 0;
39
40 // Communication Lidar
41 SoftwareSerial lidarSerial(4, 9);
42 TFMini SeeedTFMini;
43 TFLidar SeeedTFLidar(&SeeedTFMini);
44
45 // ISR pour les encodeurs
46 void ISR_countA() { counter_A++; }
47 void ISR_countB() { counter_B++; }
48
49 // Conversion cm      steps
50 int CMtoSteps(float cm) {
51     float circumference = (wheeldiameter * 3.14) / 10;
52     float cm_step = circumference / stepcount;
53     return (int)(cm / cm_step);
54 }
55
56 // Conversion degr s      steps
57 int degToSteps(float deg) {
58     float rad = deg * PI / 180.0;
59     float distanceRoue = (wheel_base / 2.0) * rad;
60     float circumference = wheeldiameter * PI;
61     float stepsPerMm = stepcount / circumference;
62     return (int)(distanceRoue * stepsPerMm);
63 }
64
65 // Lecture du LIDAR
66 void look() {
67     if (SeeedTFLidar.get_frame_data()) {
68         lidarDistance = SeeedTFLidar.get_distance();
69     }
70 }
71
72 // Fonctions de d placement
73 int MoveForward(int steps, int mspeed) {
74     counter_A = 0;
75     counter_B = 0;
76
77     digitalWrite(in1, LOW);
```

```
78  digitalWrite(in2, HIGH);
79  digitalWrite(in3, LOW);
80  digitalWrite(in4, HIGH);
81
82  while (steps > counter_A && steps > counter_B) {
83      look();
84      if (lidarDistance < 31 && lidarDistance > 0) {
85          analogWrite(enA, 0);
86          analogWrite(enB, 0);
87          return counter_A;
88      }
89      analogWrite(enA, mspeed);
90      analogWrite(enB, mspeed);
91  }
92
93  analogWrite(enA, 0);
94  analogWrite(enB, 0);
95  counter_A = 0;
96  counter_B = 0;
97  return steps;
98 }
99
100 // Les autres fonctions (MoveReverse, SpinRight, SpinLeft)...
101 // ...
102
103 // Gestion de la trajectoire
104 void suivreTrajectoire() {
105     static int stepsRestants = 0;
106     switch (segment) {
107         case 0:
108             if (stepsRestants == 0) stepsRestants = CMtoSteps(200);
109             int stepsFaits = MoveForward(stepsRestants, 150);
110             if (stepsFaits == stepsRestants) {
111                 stepsRestants = 0;
112                 segment++;
113             } else {
114                 stepsRestants -= stepsFaits;
115             }
116             break;
117         case 1:
```

```
118     currentState = FIN;
119     break;
120 }
121 }
122
123 // vitelement
124 void eviterObstacle() {
125     analogWrite(enA, 0);
126     analogWrite(enB, 0);
127     delay(100);
128
129     SpinRight(degToSteps(90), 150);
130     delay(100);
131     MoveForward(CMtoSteps(50), 150);
132     delay(100);
133     SpinLeft(degToSteps(90), 150);
134     delay(100);
135
136     currentState = TRAJECTOIRE;
137 }
138
139 // Setup et loop
140 void setup() {
141     Timer1.initialize(1000000);
142
143     pinMode(in1, OUTPUT);
144     pinMode(in2, OUTPUT);
145     pinMode(in3, OUTPUT);
146     pinMode(in4, OUTPUT);
147     pinMode(enA, OUTPUT);
148     pinMode(enB, OUTPUT);
149
150     pinMode(MOTOR_A, INPUT_PULLUP);
151     pinMode(MOTOR_B, INPUT_PULLUP);
152
153     SseedTFLidar.begin(&lidarSerial, 115200);
154     Serial.begin(9600);
155
156     attachInterrupt(digitalPinToInterrupt(MOTOR_A), ISR_countA,
        RISING);
```

```
157     attachInterrupt(digitalPinToInterrupt(MOTOR_B), ISR_countB,
158                     RISING);
159 }
160 void loop() {
161     look();
162     if (lidarDistance < 31 && lidarDistance > 0) currentState =
        EVITEMENT;
163
164     switch (currentState) {
165         case TRAJECTOIRE:
166             suivreTrajectoire();
167             break;
168         case EVITEMENT:
169             eviterObstacle();
170             break;
171         case FIN:
172             analogWrite(enA, 0);
173             analogWrite(enB, 0);
174             break;
175     }
176 }
177
178 void Timer_Isr() {
179     Timer1.detachInterrupt();
180     Timer1.attachInterrupt(Timer_Isr);
181 }
```